Introduction to

Information Retrieval

Course Outline

Course No	***Course Title***	***Hrs / week***		***Year***	***Semester***	**Exam Hours**
		Lect	***Lab***			
IS418	Information Storage and Retrieval	2	2	2026/ 2027 – 4th Year	First	2

Assessments Methods:

Assessment	weight
Midterm Exam	20%
Oral Examination & Lab	10%
Practical Examination	10%
Final-term Examination	60%
Total	100 %

Course Resources

- **Textbook :**
  - Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze “An Introduction to Information Retrieval”, Cambridge University Press, Cambridge, England, 2009

- **Additional Materials:**
  - Lecture Slides.

Course Content

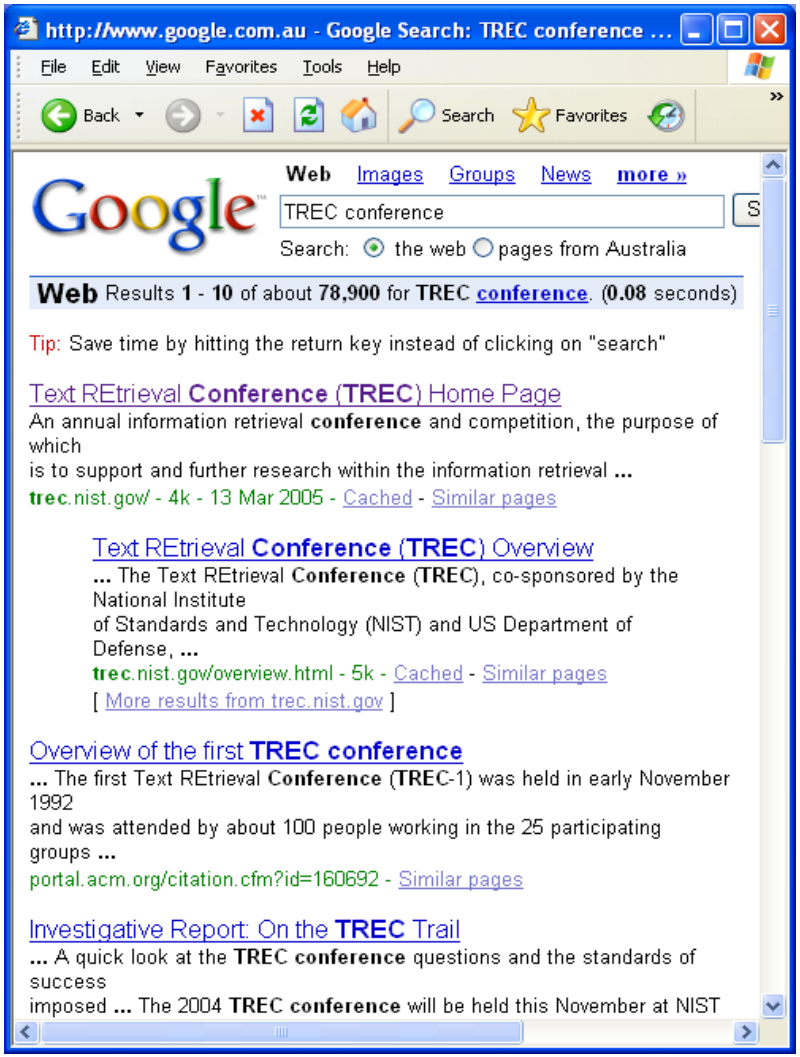
- **Chap. 1**: Introducing Information Retrieval and Web Search
- **Chap. 2**: The term vocabulary and postings lists
- **Chap. 3**: Dictionaries and tolerant retrieval
- **Chap. 6**: Scoring, term weighting and the vector space mode
- **Chap. 8**: Evaluation in information retrieval

Introduction to

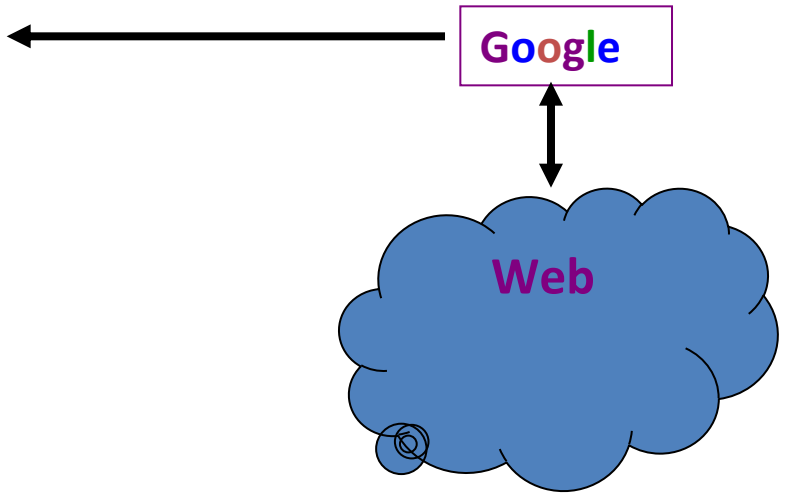
Information Retrieval

Introducing Information Retrieval and Web Search

Introduction





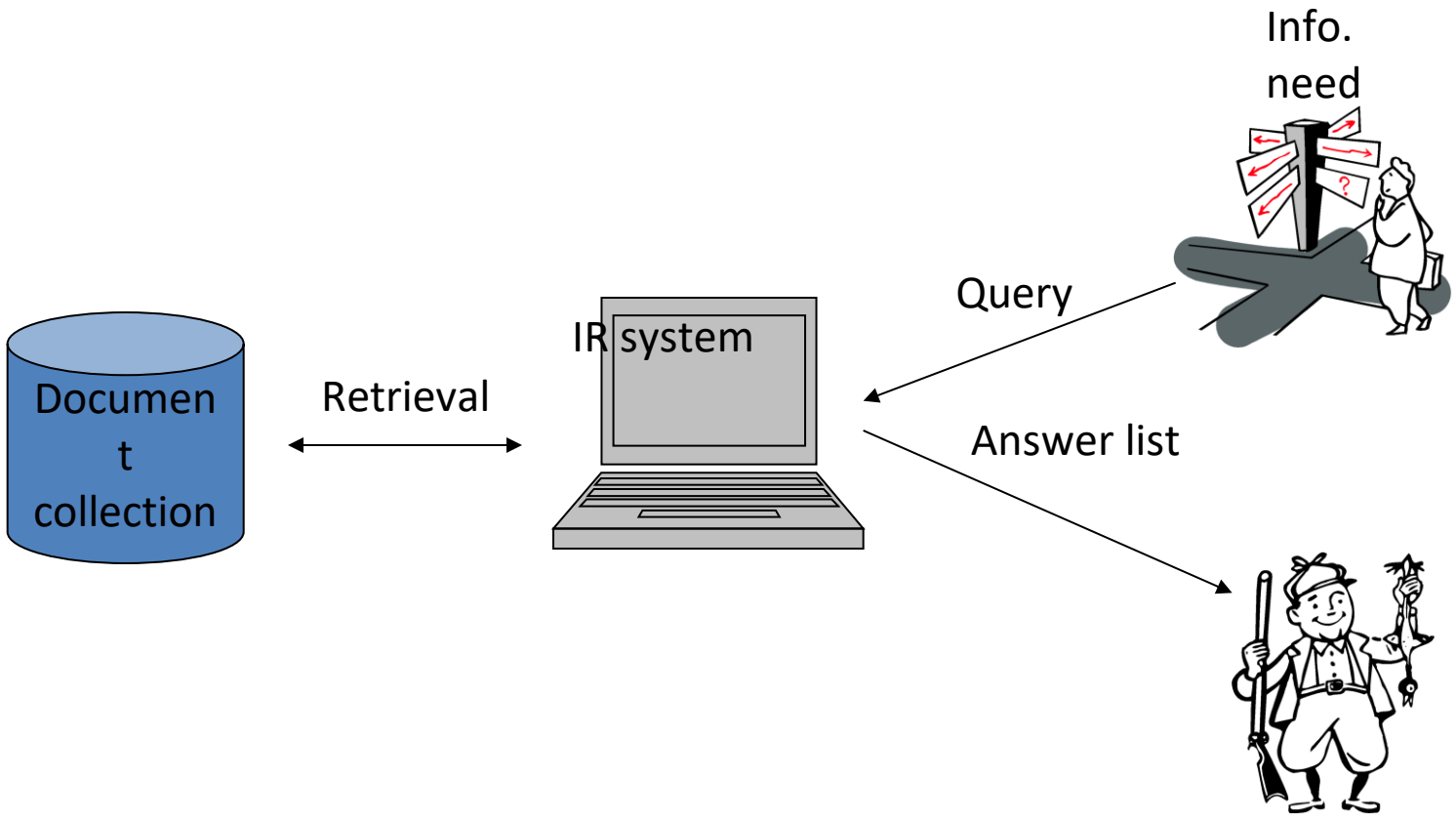


Basic assumptions of Information Retrieval

- **Collection**: A set of documents over which we perform retrieval
  - Sometimes referred to as a **corpus**
  - Assume it is a static collection for the moment
- **Information Need** is the topic about which the user **desires to know** more and is differentiated from a query
- **Query** is what the user conveys to the computer in an attempt to communicate the information need.
- **Relevance** :a document is **relevant** if it is one that the user perceives as **containing information** of value with respect to their personal information need.

The problem ???

- **Goal** = find documents *relevant* to the user's information need from a large document set



Information Retrieval

- **Information Retrieval** (IR) is finding material (usually documents) of an unstructured nature (usually text) that satisfies an information need from within large collections (usually stored on computers).
  - These days we frequently think first of web search, but there are many other cases:
    - E-mail search
    - Searching your laptop
    - Legal information retrieval

Possible Approaches of Information Retrieval

- **Grep**: the simplest form of document retrieval is for a computer to do this sort of linear scan through documents (is called grepping through text)

  - Grep is a Unix command which perform this process

- **String matching (linear search in documents)**
  - Slow
  - Difficult to improve

Main issues in IR

- **Query evaluation** (or retrieval process)
  - To what extent does a document correspond to a query?
- **System evaluation**
  - How good is a system?
  - Are the retrieved documents relevant? (precision)
  - Are all the relevant documents retrieved? (recall)

How good are the retrieved docs?

- **Effectiveness**: to assess the effectiveness of an IR system (the quality of its search results)
- A user usually wants to know two key statistics about the system's results for a query
  - ***Precision*** : Fraction of retrieved docs that are relevant to the user's information need
  - ***Recall*** : Fraction of relevant docs in collection that are retrieved
- More precise definitions and measurements to follow later

Introduction to

Information Retrieval

Structured vs. Unstructured Data

IR vs. databases: Structured vs unstructured data

- Structured data tends to refer to information in “tables”

Employee	Manager	Salary
Smith	Jones	50000
Chang	Smith	60000
Ivy	Smith	50000

Typically allows numerical range and exact match (for text) queries,
e.g.,

Salary < 60000 AND Manager = Smith.

Unstructured data

- Typically refers to free text
- Allows
  - Keyword queries including operators
  - More sophisticated “concept” queries e.g.,
    - find all web pages dealing with *drug abuse*
- Classic model for searching text documents

Semi-structured data

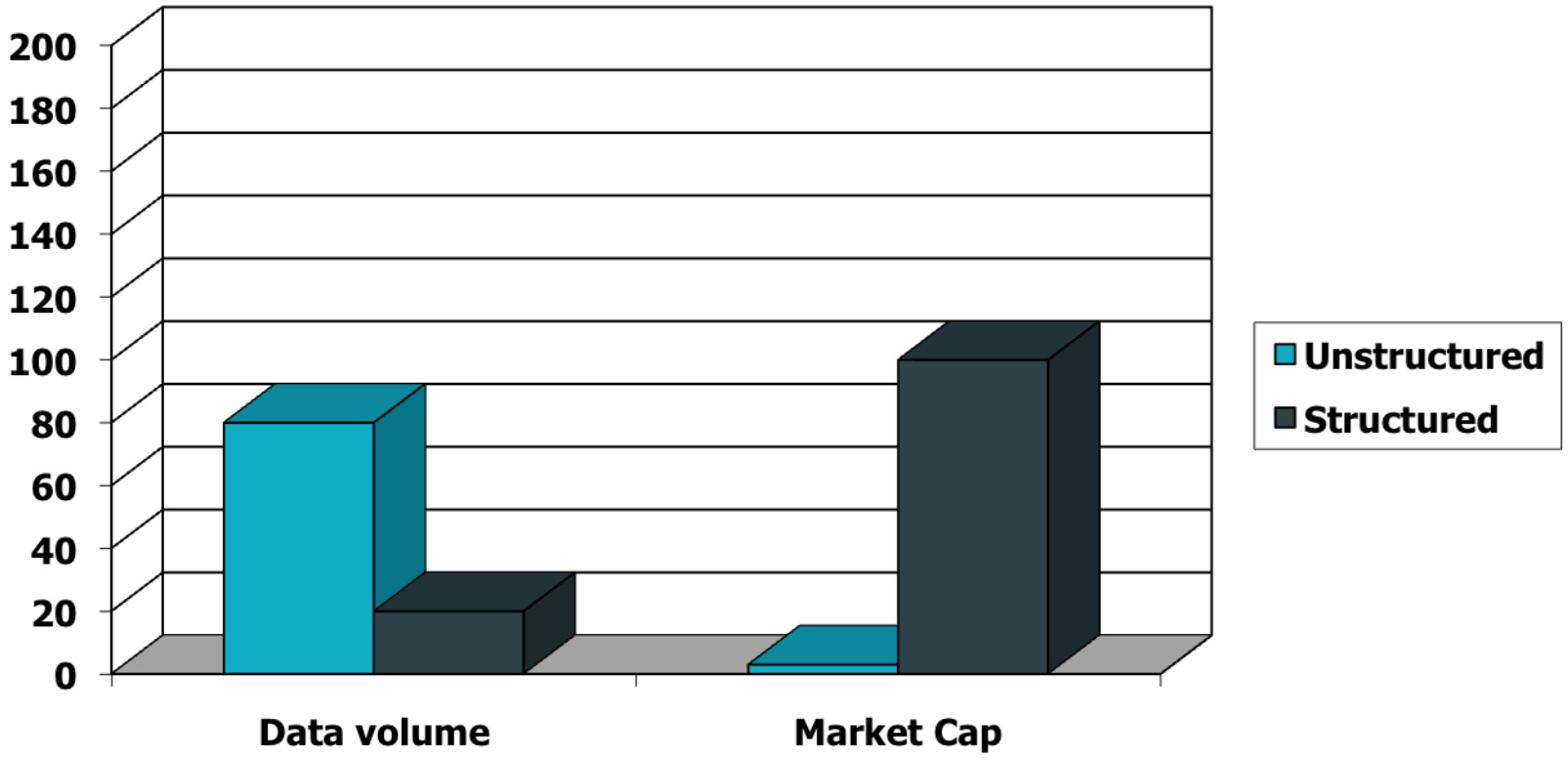
- In fact, almost no data is "unstructured"
- E.g., this slide has distinctly identified zones such as the *Title* and *Bullets*
    - … to say nothing of linguistic structure
- IR is also used to facilitates "semi-structured" search such as
    - *Finding a document where the*

      Title contains data AND *Bullets* contain search

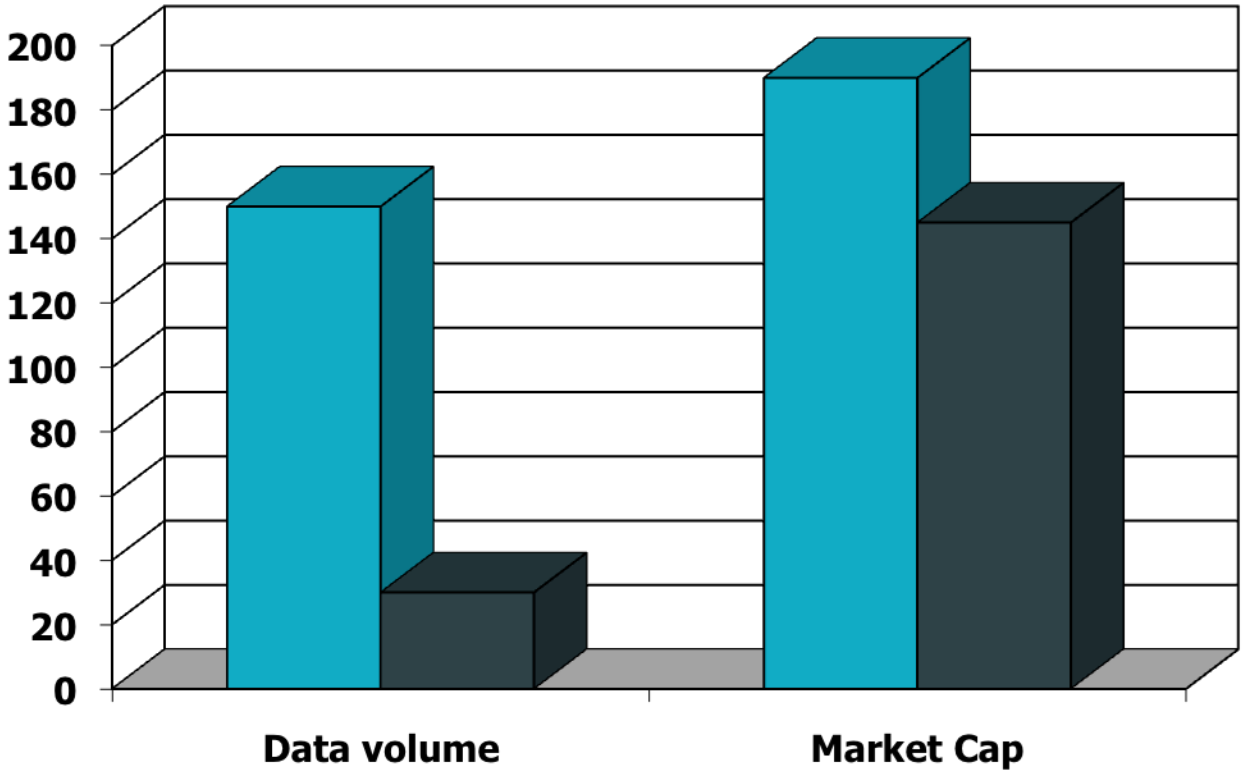
      Title contains Java AND *Body* contain threading
- Or even

  - *Title* is about Object Oriented Programming AND *Author* something like stro*rup
    - where * is the wild-card operator

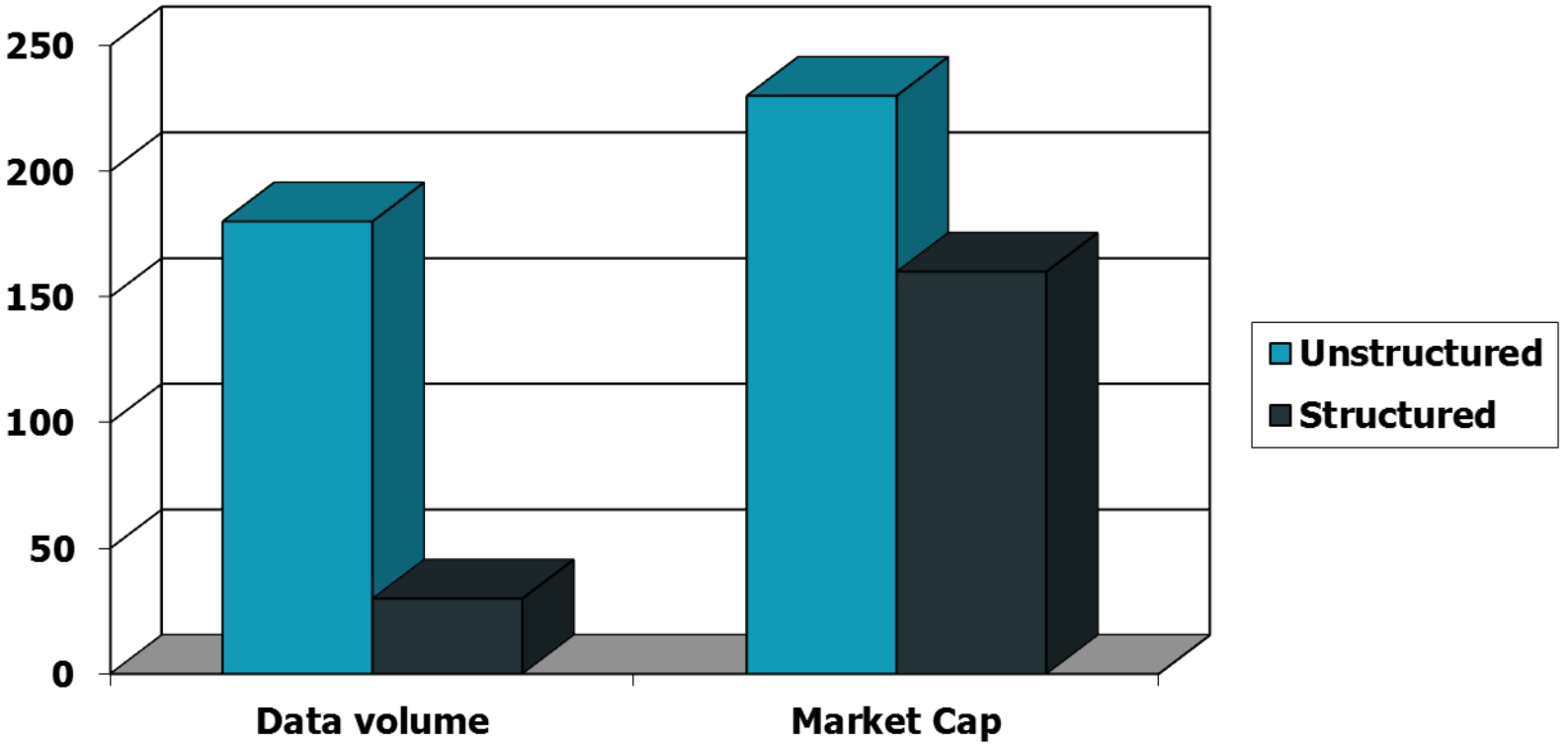
Unstructured (text) vs. structured (database) data in 1996



Unstructured (text) vs. structured (database) data in 2009



Unstructured (text) vs. structured (database) data today



Introduction to

Information Retrieval

Term-document incidence matrices

An example information retrieval problem

- Which plays of Shakespeare contain the words ***Brutus*** *AND* ***Caesar*** but *NOT* ***Calpurnia***?
- One could grep all of Shakespeare's plays for ***Brutus*** and ***Caesar,*** then strip out lines containing ***Calpurnia***?
- Why is that not the answer?
  - Slow (for large corpora)
  - *NOT* ***Calpurnia*** is non-trivial
  - Other operations (e.g., find the word ***Romans*** near ***countrymen***) not feasible
  - Ranked retrieval (best documents to return)

An example information retrieval solution

- The way to avoid linearly scanning the texts for each query is to INDEX the documents in advance.
- **Index** is used to introduce the basics of the Boolean retrieval model

Boolean retrieval model

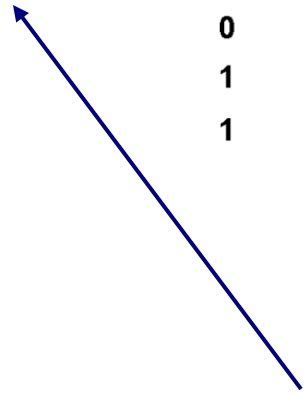
- Suppose we record each document (a play of Shakespeare's) whether it contains each word out of all the words Shakespeare used (Shakespeare used about 32,000 different words)
- **Boolean Retrieval Model**: is a model for information retrieval in which we can pose any query which is in the form of a Boolean expression of terms, in which terms are combined with operators AND,OR, and NOT.
  - The model views each document as a set of words
- The result is a binary term-document "incidence matrix"
- **Terms** are the indexed units.

- Terms are usually words

- some of terms phrase

Term-document incidence matrices

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0



***Brutus** AND **Caesar** BUT NOT **Calpurnia** ???*

1 if play contains word, 0 otherwise

Incidence vectors

- So we have a 0/1 vector for each term.
- To answer query: take the vectors for ***Brutus, Caesar*** and ***Calpurnia*** (complemented the last) ⇨ bitwise *AND*.
  - **110100 *AND* 110111 *AND* 101111 = 100100**

Answers to query

- Antony and Cleopatra, Act III, Scene ii

Agrippa [Aside to DOMITIUS ENOBARBUS]: Why, Enobarbus,
When Antony found Julius ***Caesar*** dead,
He cried almost to roaring; and he wept
When at Philippi he found ***Brutus*** slain.

- Hamlet, Act III, Scene ii

Lord Polonius: I did enact Julius ***Caesar*** I was killed i' the Capitol; ***Brutus*** killed me.

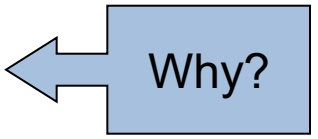


Bigger collections

- Consider corpus has *N* = 1 million documents,
- Each document with about 1000 words.
- Each word Average 6 bytes/word including spaces/punctuation
- Size of corpus= 1million X 1000 X 6 = 6GB
- Number of distinct terms *M* = 500,000 *distinct* terms among these documents.
- Number of cells in the term-document matrix=1 million X 500,000= .5 trillion (too much for memory),
- Can we cut down on the space?

Can't build the term-document matrix

- A 500K x 1M matrix has half-a-trillion 0's and 1's (at most .2% of the cells can have a 1).
- Too many to fit computer's memory.
- But it has no more than one billion 1's.
  - matrix is extremely sparse.
  - Minimum of 99.8% of the cells are zero.
- What's a better representation?
  - Is to record only the things that do occur
  - We only record the 1 positions.



Introduction to

Information Retrieval

The Inverted Index

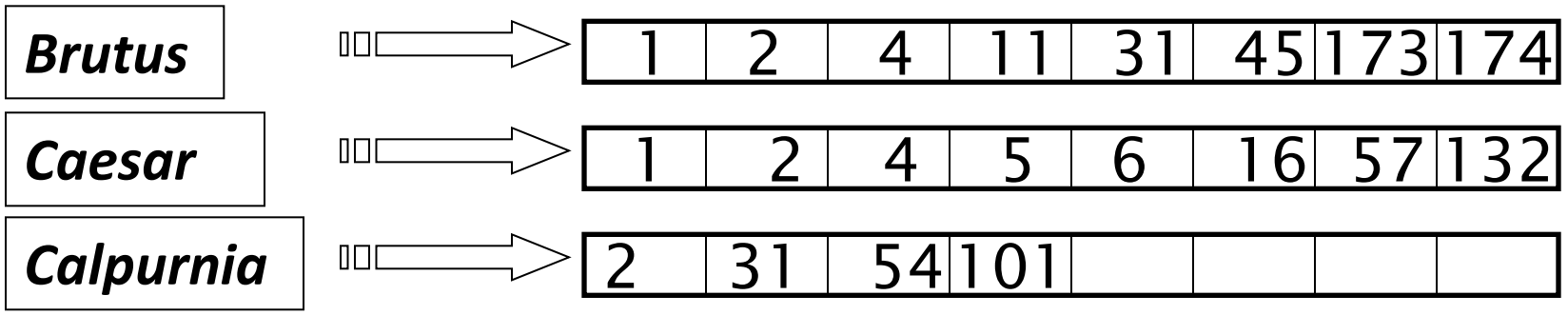
The key data structure underlying modern IR

Inverted index

- It sometimes called inverted file
- It keeps a dictionary of terms (sometimes referred to as vocabulary or lexicon).
- We use dictionary for the data structure and vocabulary for the set of terms
- **Posting list (inverted list):** a list that records which documents the terms occurs in
- All the postings lists taken together are referred to as the **postings**
- **Posting (position in the document)**: each item in the list which records that a term appeared in a document.
- The dictionary is sorted alphabetically and each postings list is sorted by document ID

Inverted index

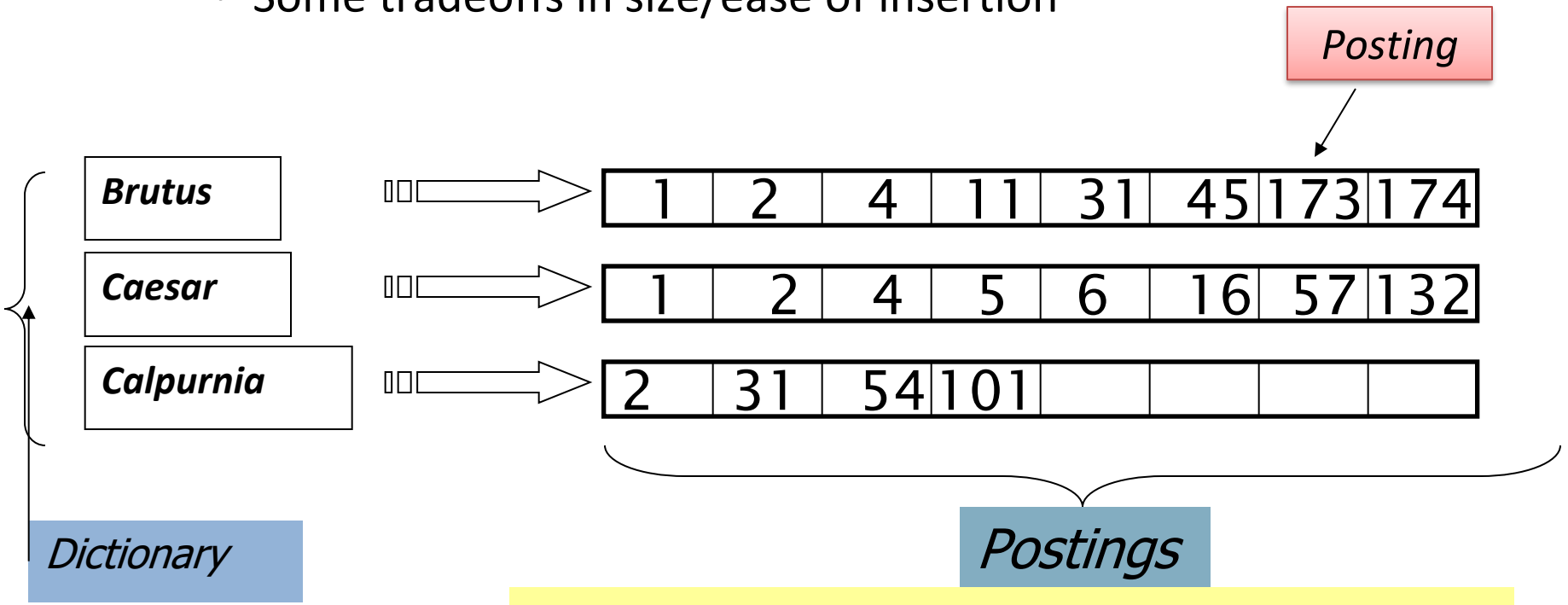
- For each term *t*, we must store a list of all documents that contain *t*.
  - Identify each doc by a **docID**, a document serial number
  - DocID a unique number for each document, known as the document identifier
- Can we used fixed-size arrays for this?



What happens if the word ***Caesar*** is added to document 14?

Inverted index

- We need variable-size postings lists
  - On disk, a continuous run of postings is normal and best
  - In memory, can use linked lists or variable length arrays
    - Some tradeoffs in size/ease of insertion

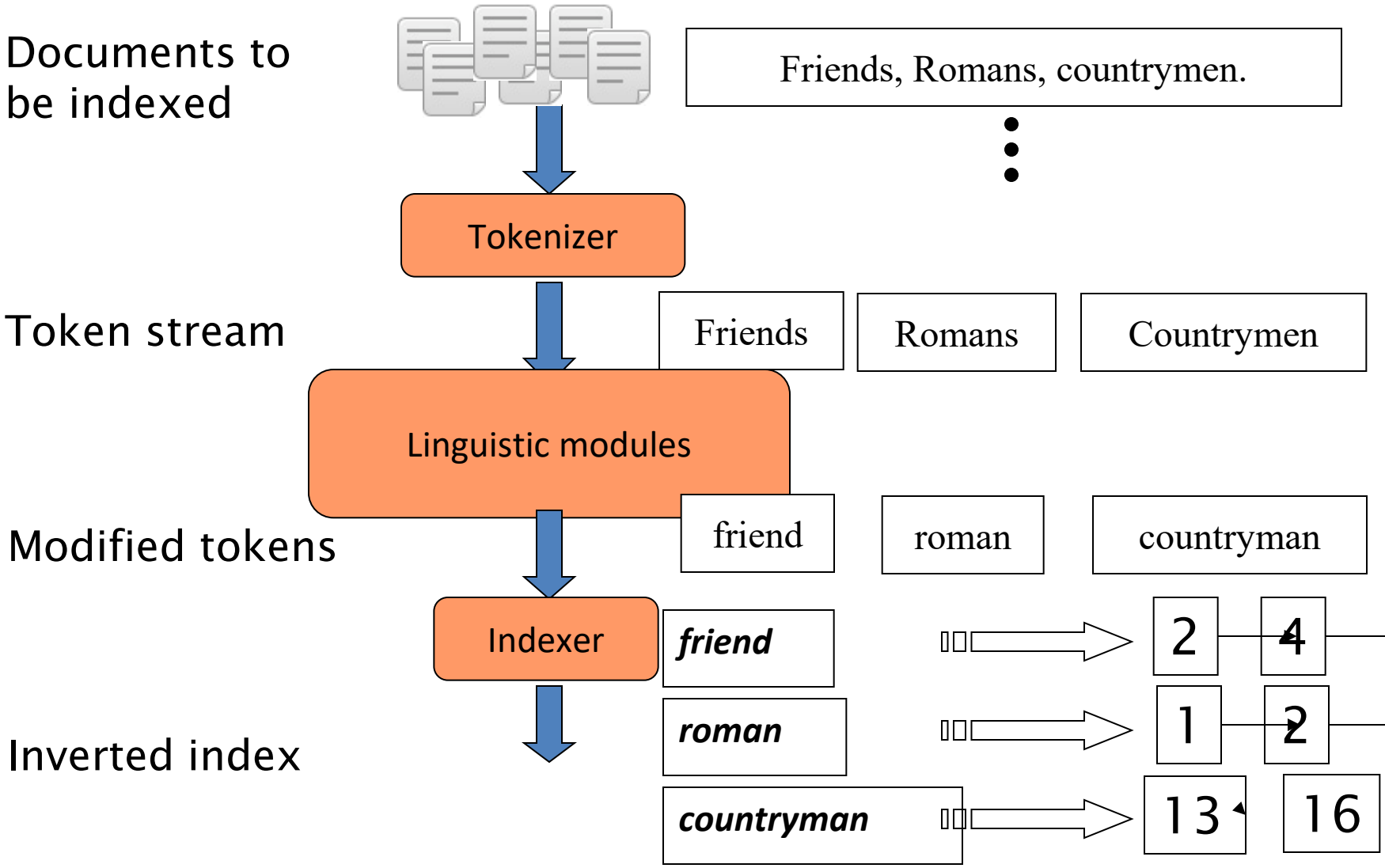


Sorted by docID (more later on why).

Inverted index construction

1. Collect the documents to be indexed
2. Tokenize the text, turning each document into a list of tokens
3. Do linguistic preprocessing, producing a list of normalized tokens, which are the indexing terms
4. Index the documents that each term occurs in by creating an inverted index, consisting of a dictionary and postings

Inverted index construction



Initial stages of text processing

- Tokenization
  - Cut character sequence into word tokens
    - Deal with ***"John's"***, ***a state-of-the-art solution***
- Normalization
  - Map text and query term to same form
    - You want ***U.S.A.*** and ***USA*** to match
- Stemming
  - We may wish different forms of a root to match
    - ***authorize***, ***authorization***
- Stop words
  - We may omit very common words (or not)
    - ***the, a, to, of***

Indexer steps: Token sequence

- Sequence of (Modified token, Document ID) pairs.

Doc 1

I did enact Julius Caesar I was killed i' the Capitol; Brutus killed me.

Doc 2

So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious



Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Indexer steps: Sort

- Sort by terms
  - And then docID

Core indexing step

Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

Indexer steps: Dictionary & Postings

- Multiple term entries in a single document are merged.
- Split into Dictionary and Postings
- Doc. frequency information is added.
- Doc. Frequency: the number of documents which contain each term (which is the length of each postings list)

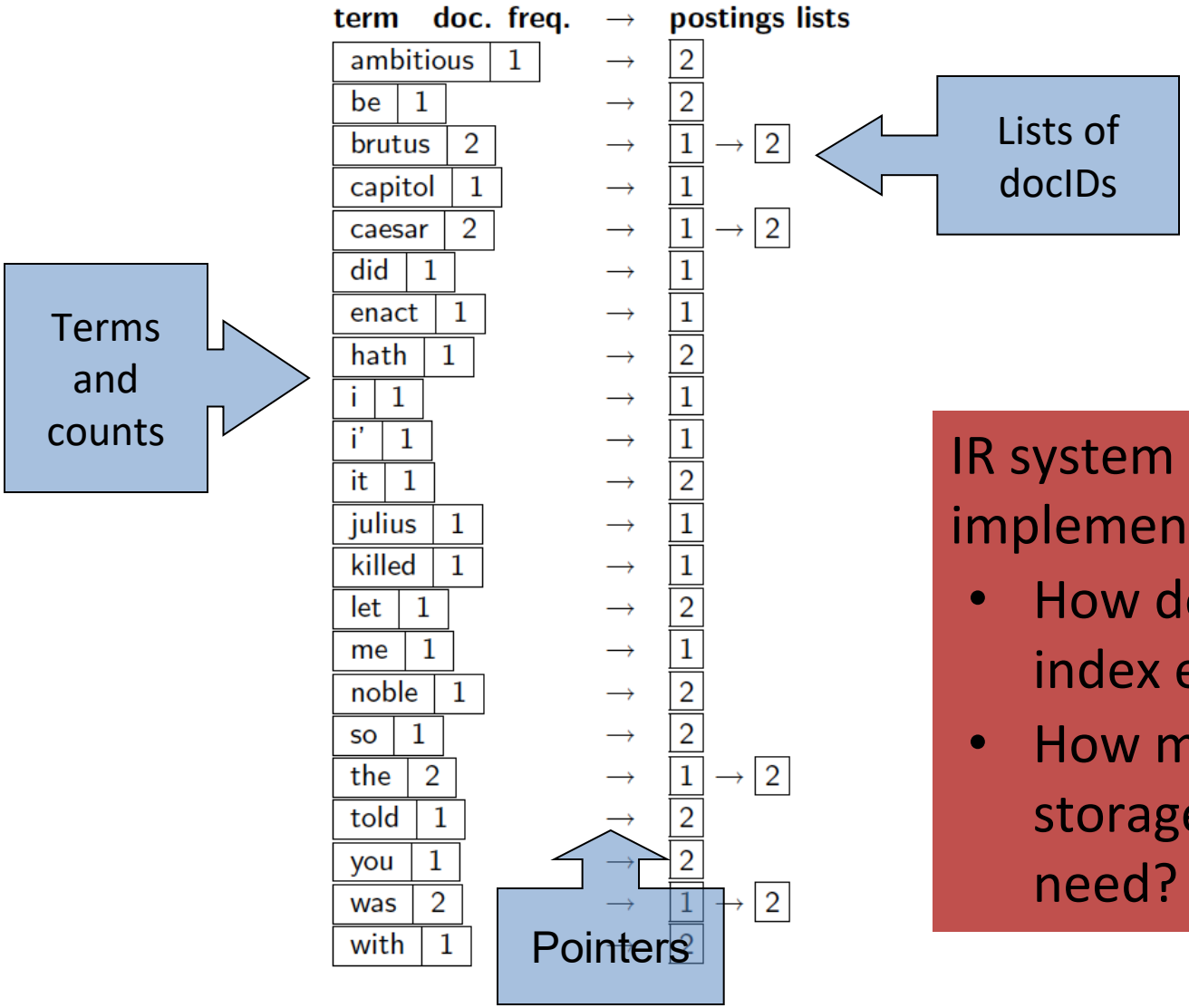
Why frequency?
Will discuss later.

Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2



term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
i	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2

Where do we pay in storage?



IR system implementation

- How do we index efficiently?
- How much storage do we need?

What data structure should be used for posting list?

- A fixed length array would be wasteful, some words occur in many documents .
- Two good alternatives are linked lists or variable length arrays.
- We can use a hybrid scheme , with linked list of fixed length arrays for each term.